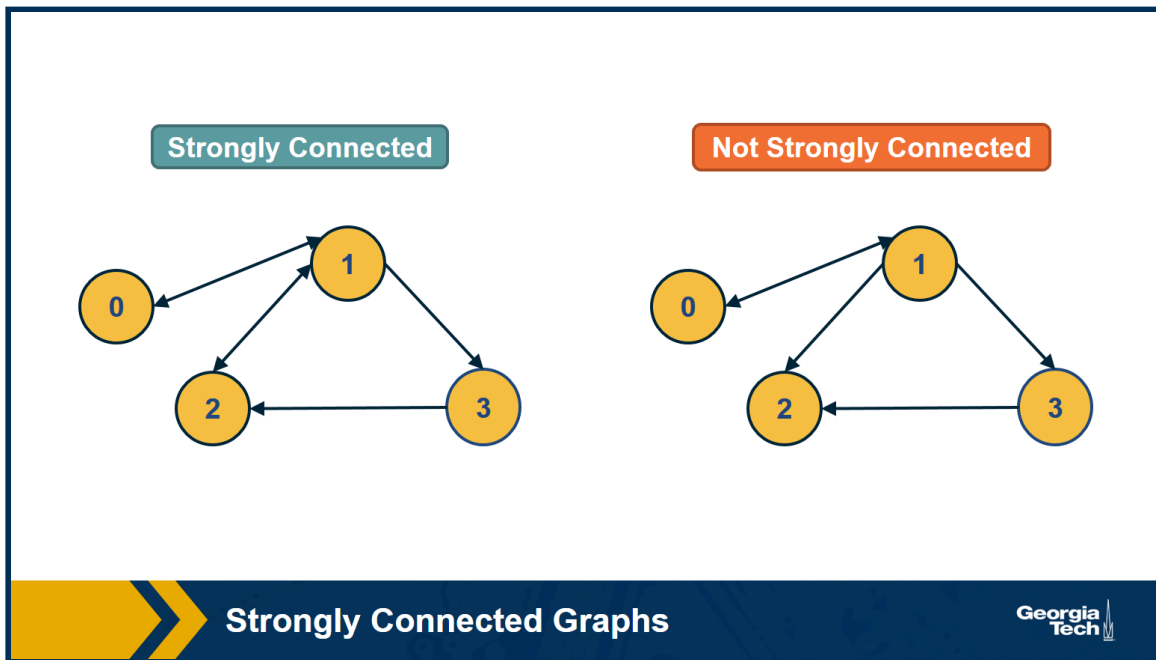


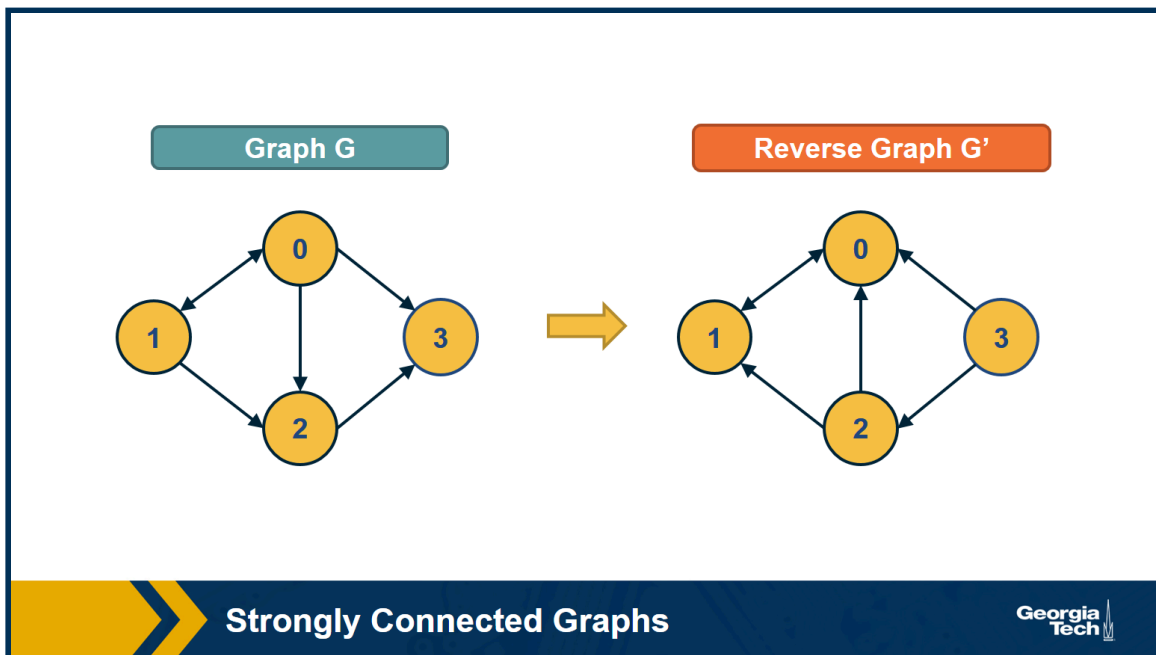
L2: Strongly Connected Components

In directed graphs, the notion of connectedness is different: a node s may be able to reach a node t through a (**directed**) path – but node t may not be able to reach node s .

A directed graph is **strongly connected** if there is a path between all pairs of vertices. A **strongly connected component** (SCC) of a directed graph is a maximal **strongly connected** subgraph.

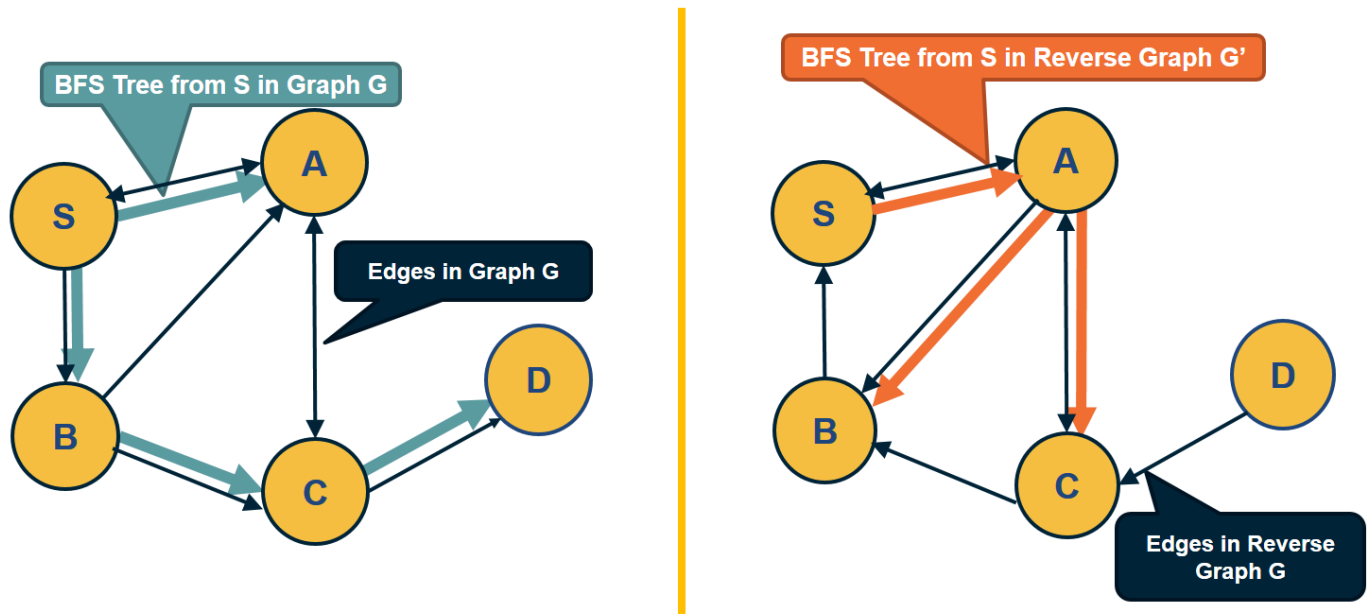


If the graph has only one SCC, we say that it is strongly connected. **How would you check (in linear time) if a directed graph is strongly connected?** Please think about this for a minute before you see the answer below.



Answer

First, note that a directed graph is strongly connected if and only if *any* node s can reach all other nodes, and *every other* node can reach s . So, we can pick any node s and then run BFS twice. First, on the original graph G . Second, run BFS on the graph G' in which each edge has opposite direction than in G -- G' is called the **reverse** graph of G . If both BFS traversals reach all nodes in G , it must be that G is strongly connected (do you see why?).



BFS Tree

The visualization above shows an example in which node D cannot reach S (so S cannot reach D in the reverse graph).

How would you compute the set of strongly connected components in a directed graph? Two famous algorithms to do so are Tarjan's algorithm and Kosaraju's algorithm. They both rely on Depth-First Search traversals and run in $\Theta(n+m)$ time, if the graph is represented with an adjacency list.

Food for Thought

We suggest you study Tarjan's or Kosaraju's algorithm. For instance, Kosaraju's algorithm is described at:

https://en.wikipedia.org/wiki/Kosaraju%27s_algorithm → https://en.wikipedia.org/wiki/Kosaraju%27s_algorithm